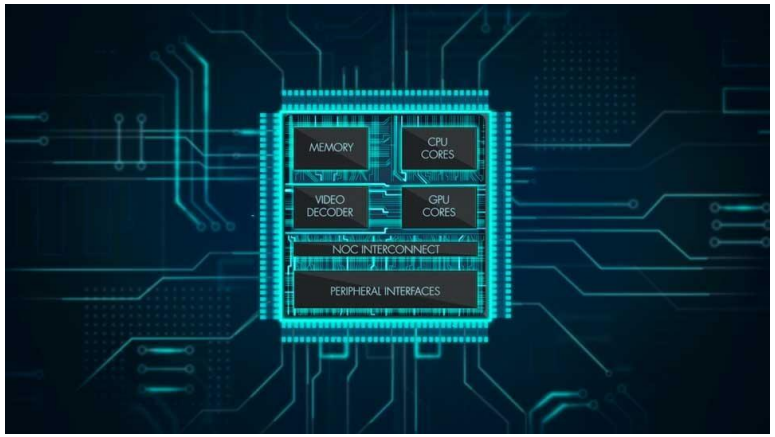
Digital System Design

Basic Operation of 2D Convolution



Date : 2026. 04. 23.

Instructor : **Prof. Hyun Kim**

Lecture Note : eclass.seoultech.ac.kr

Office : Mirae Hall #526

Web : idsl.seoultech.ac.kr

Contact : hyunkim@seoultech.ac.kr / 010-9600-5427

Contents

- Conv2d
- Conv2d Dataflow

Contents

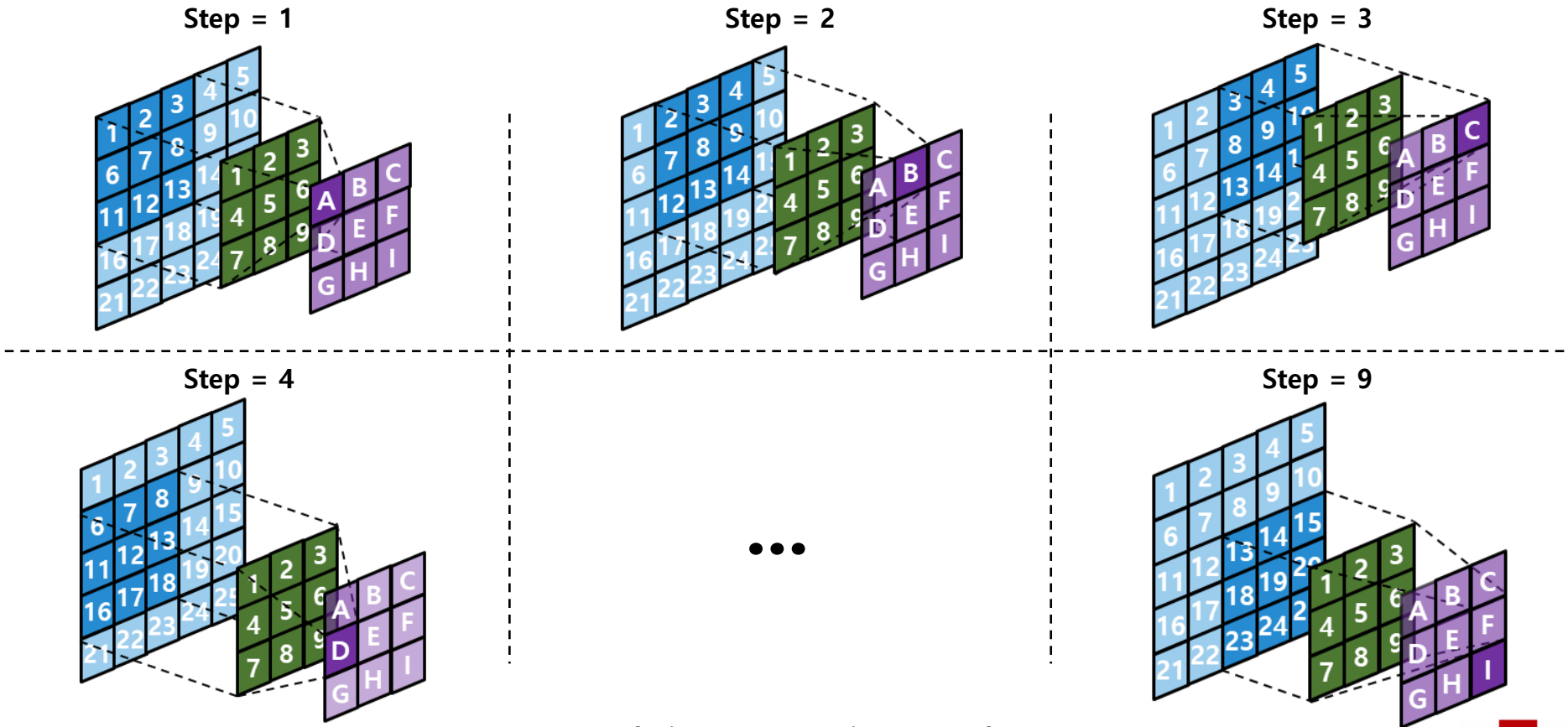
- Conv2d

- Conv2d Dataflow

Convolution 2D

What is Convolution 2D?

- An operation that extracts **local features** from 2D input data.
- Performs computation by **sliding a 2D kernel** over local regions of the input.
- Each output element is computed as a **dot product between input and kernel.**
- Produces an output in the form of a **feature map.**



<Convolution 2D Operation Example>

Contents

- Conv2d

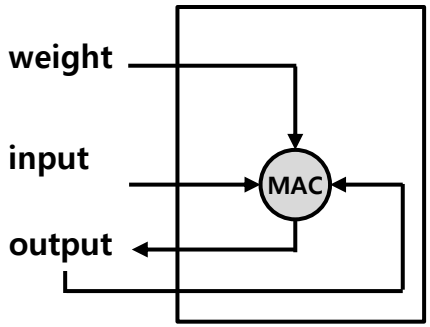
- Conv2d Dataflow

Conv2d Dataflow

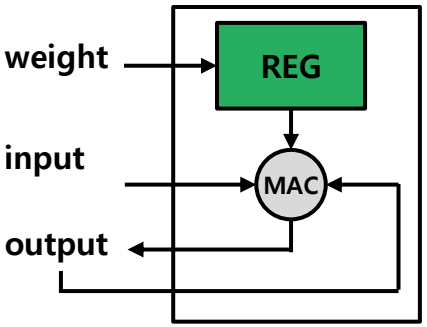
Conv2d Dataflow

- Data movement dominates energy consumption.
- Data movement affects **latency and resource utilization**.
- Dataflow determines which data stays in the PE register.
- Maximizing data reuse is key to minimizing data movement.

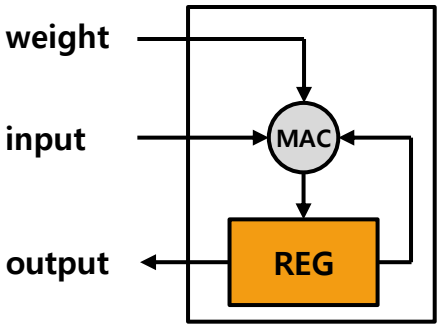
Streaming Dataflow	Weight Stationary	Output Stationary
Don't use registers in PE. Input & weight & partial sums stream through.	**Weights remain in PE registers.** Input & partial sums stream through.	**Partial sums remain in PE registers.** Weight & inputs stream through.



<Streaming PE>



<Weight reuse PE>



<Output reuse PE>

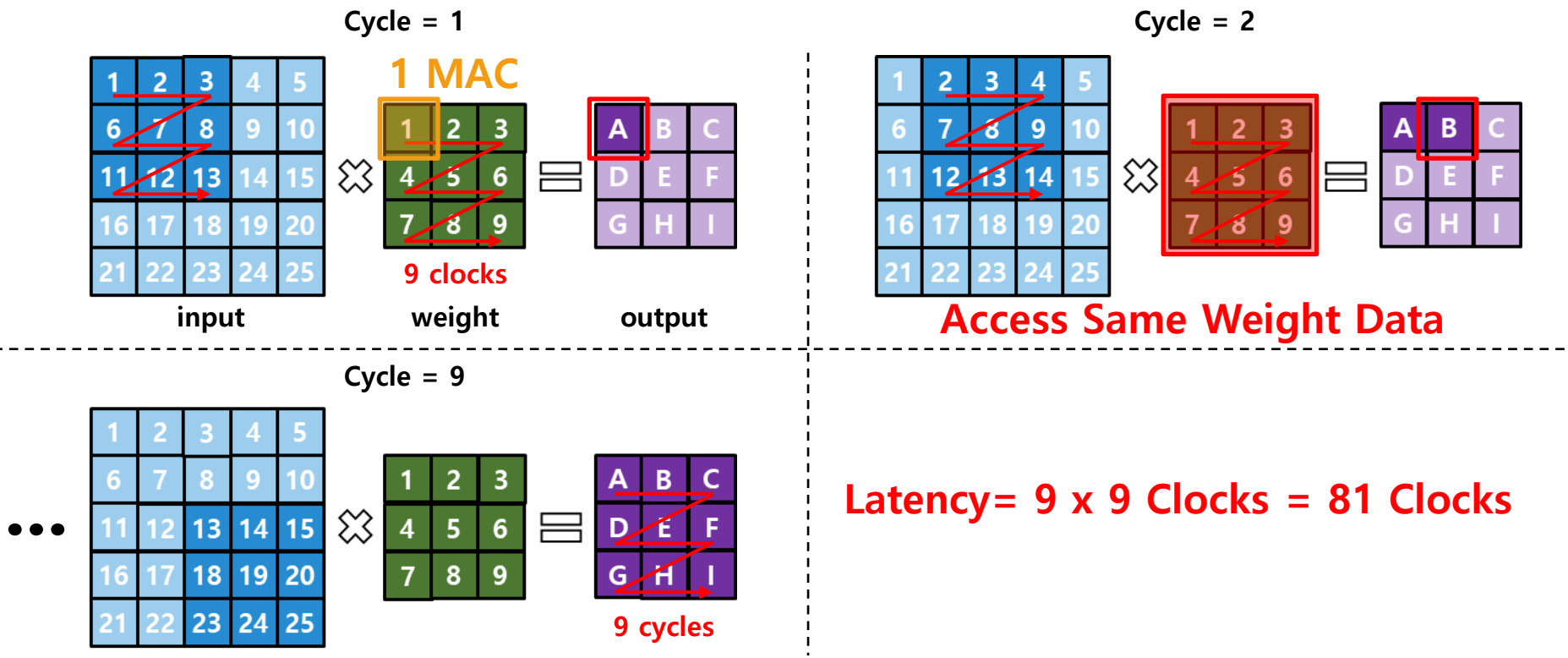
Streaming Dataflow

Streaming Dataflow (1MAC)

- Input, weight, output data is not stored and streams through.
- All data must be read from BRAM.
- Repeated access to the same weight data leads to inefficiency.
- **Output data is produced every 9 clocks.**
- **All outputs are completed within 9 cycles.**

: dataflow

: PE



Latency= 9 x 9 Clocks = 81 Clocks

<Conv2d Streaming Dataflow>

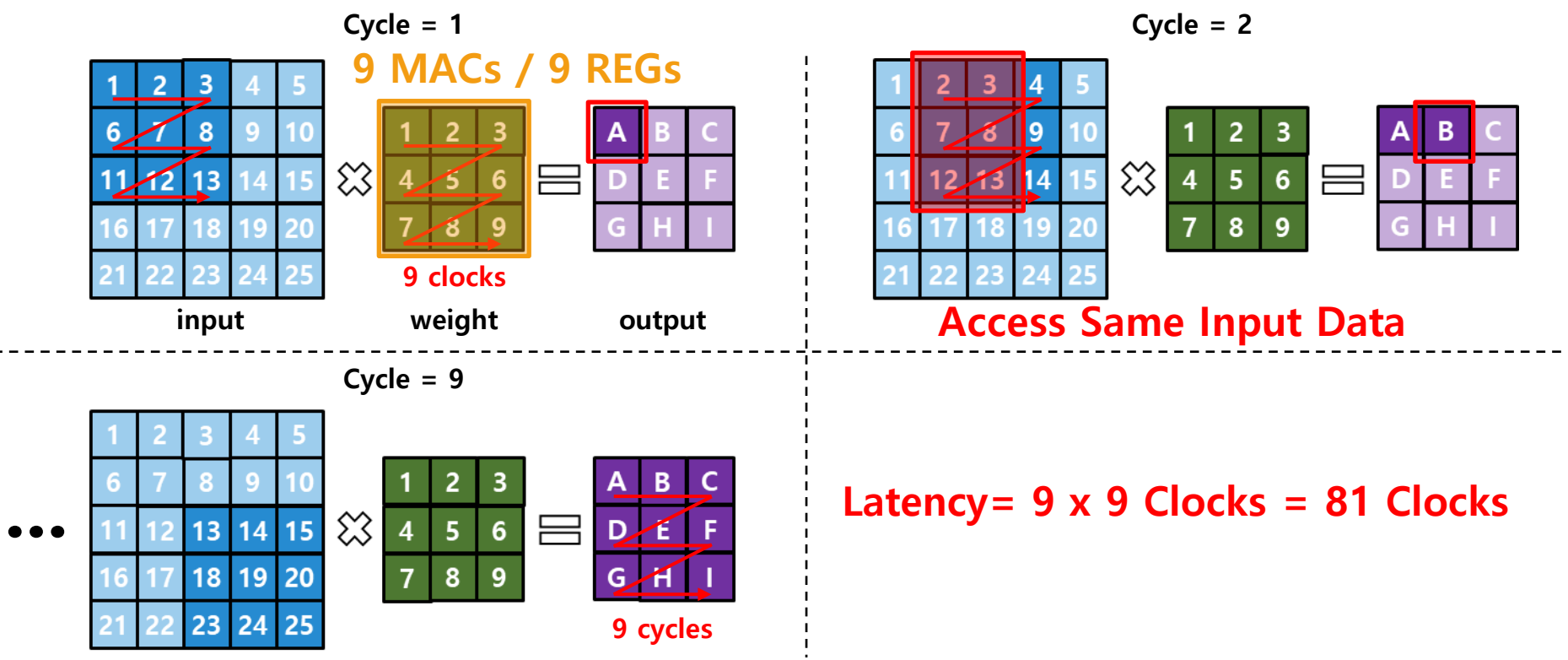
Weight Stationary (1)

Weight Stationary (9MACs/9REGs)

- **Weight data is stored in PE register.**
- Input, output data is not stored and streams through.
- Input data still needs to be fetched one by one.
- **Overall latency remains unchanged.**

: dataflow

: PE

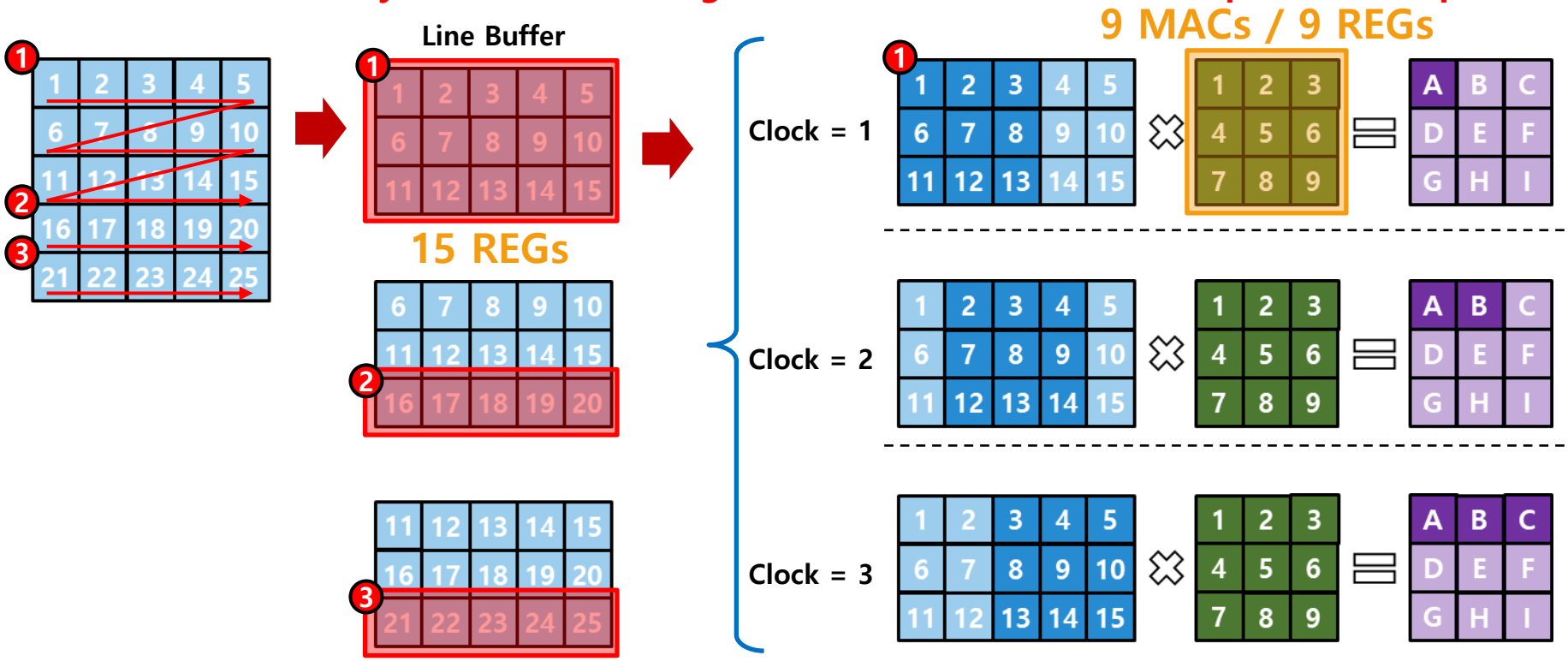


<Conv2d Weight Stationary>

Weight Stationary (2)

Weight Stationary (9MACs/9REGs) with Line Buffer (15REGs)

- Input data is fetched only once(25 clocks).
- Weight size determines the number of line buffer and PE for single-cycle computation.
- Line buffer enables parallel data access.
- **Overall latency is reduced through fewer BRAM accesses and parallel computation.**



25 Clocks + 3 Clocks x 3 Cycles = 34 Clocks **9 MACs / 9+15 REGs**

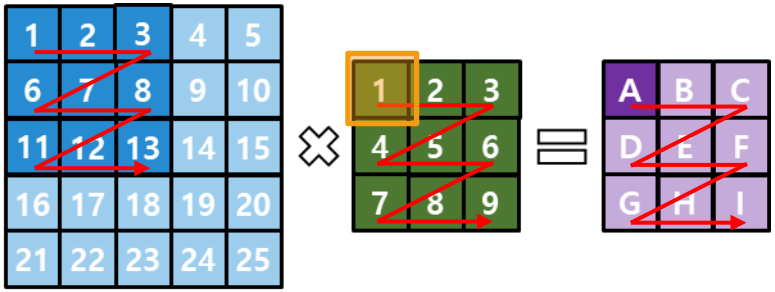
<Conv2d Weight Stationary with Line Buffer>

Streaming vs Weight Stationary

Streaming vs Weight Stationary with Line Buffer

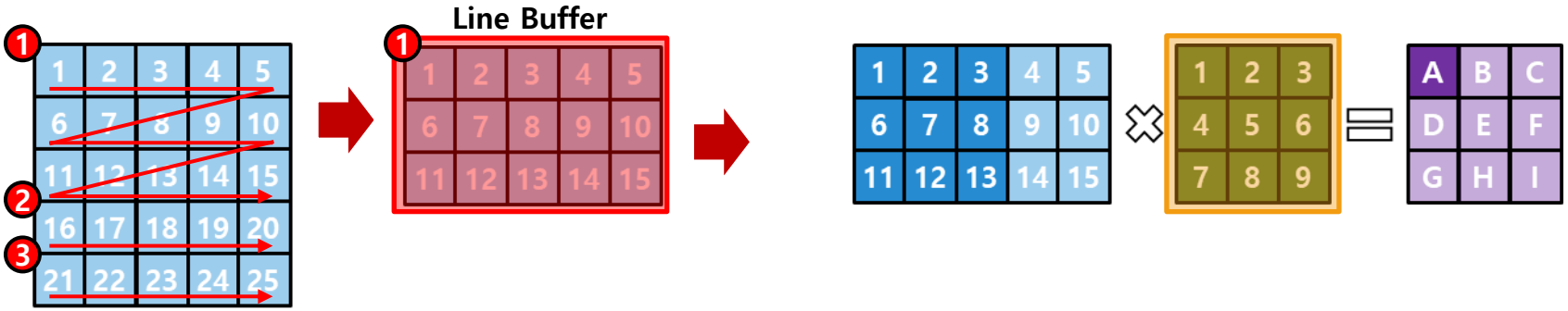
: dataflow

: PE



81 Clocks **1 MAC / 1 REG**

<Conv2d Streaming Dataflow>



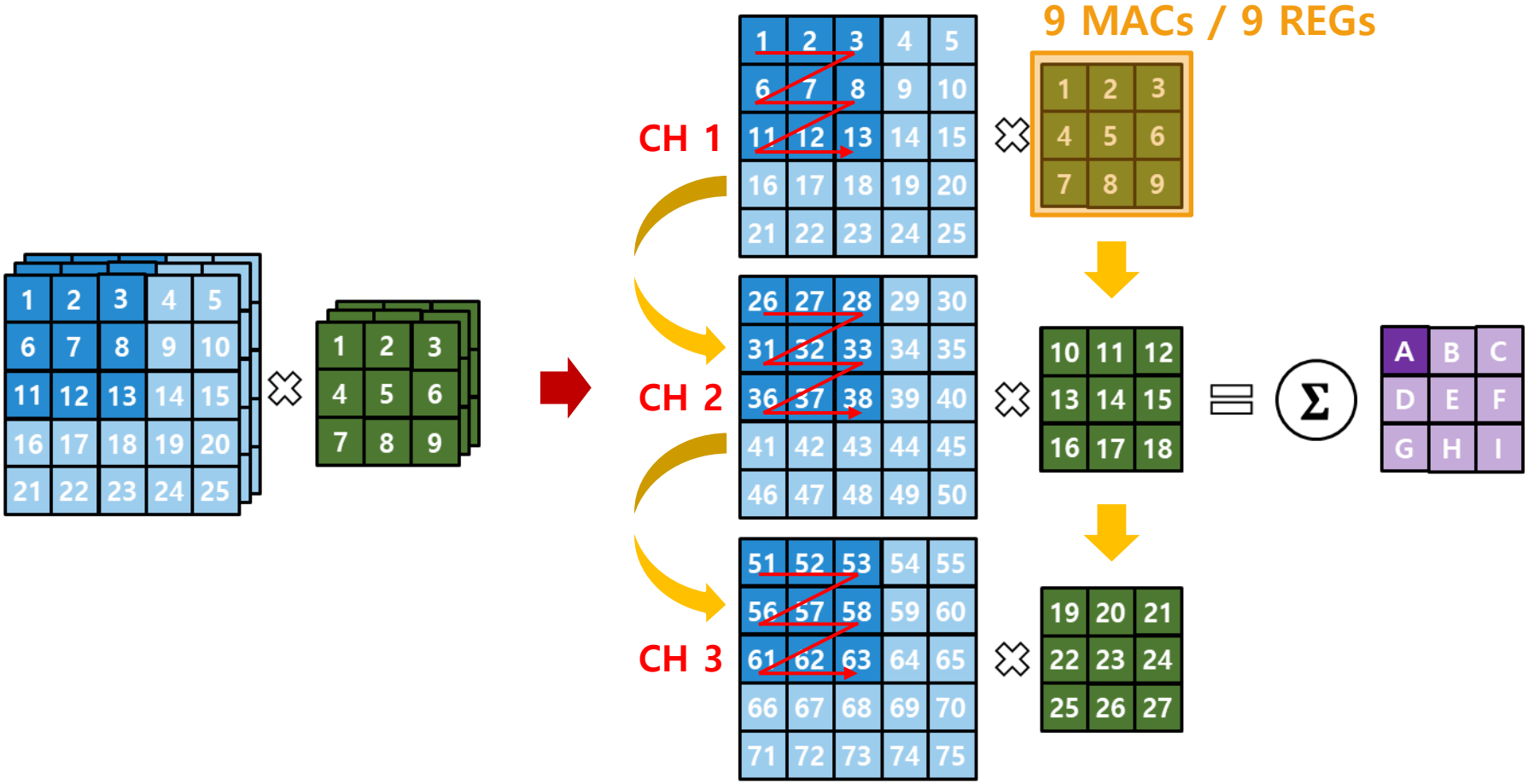
34 Clocks **9 MACs / 24 REGs**

<Conv2d Weight Stationary with Line Buffer>

Output Stationary (1)

Naive Output Stationary in Multi-channels(9MACs/9REGs)

- **Output data is stored in PE register.**
- Input and weight data is not stored and streams through.
- Partial sums for each channel are accumulated.

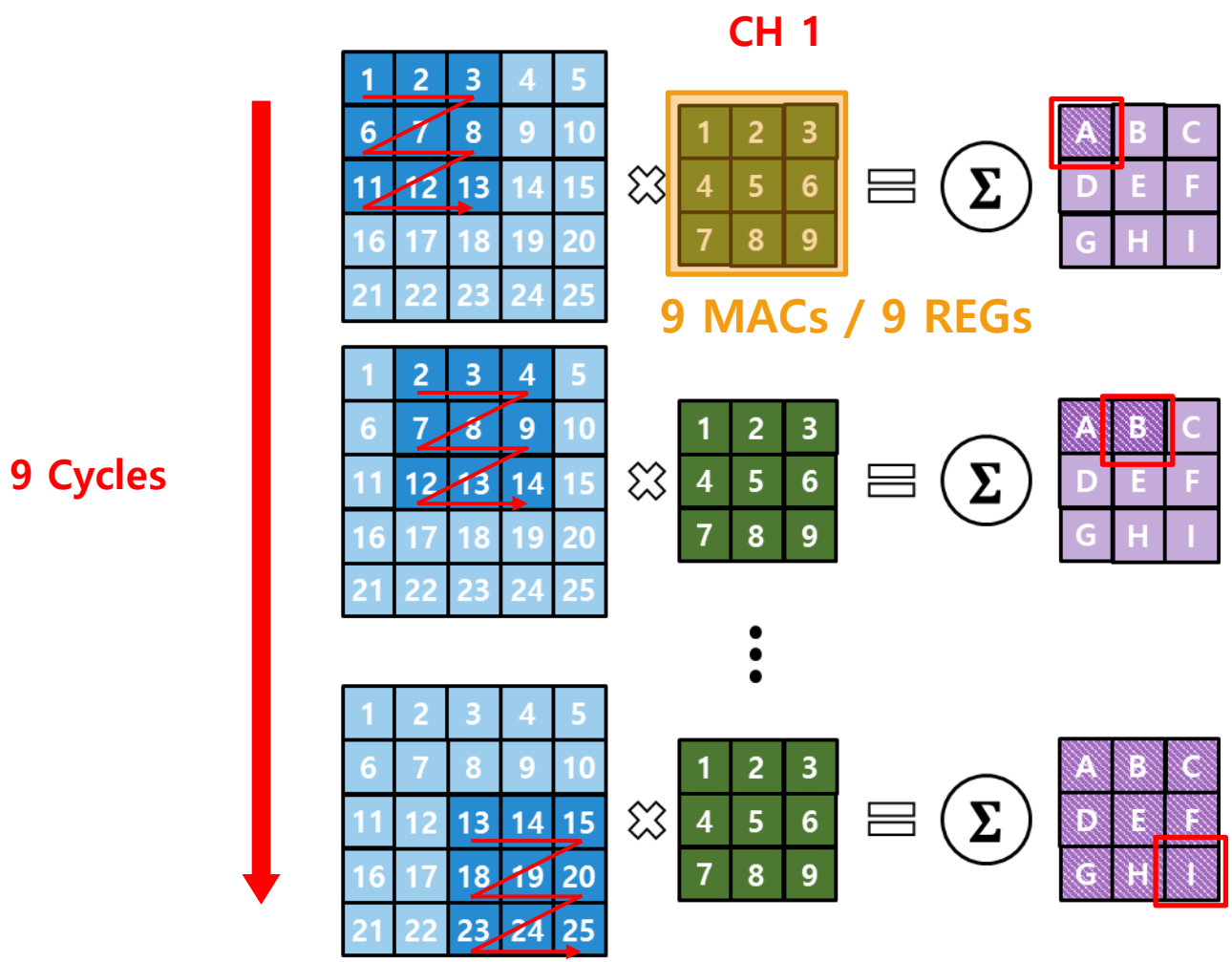


<Conv2d Naive Output Stationary>

Output Stationary (2)

◈ Naive Output Stationary(9MACs/9REGs)

A : Not Started
A : Partial Sum CH1
A : Partial Sum CH2
A : Partial Sum CH3

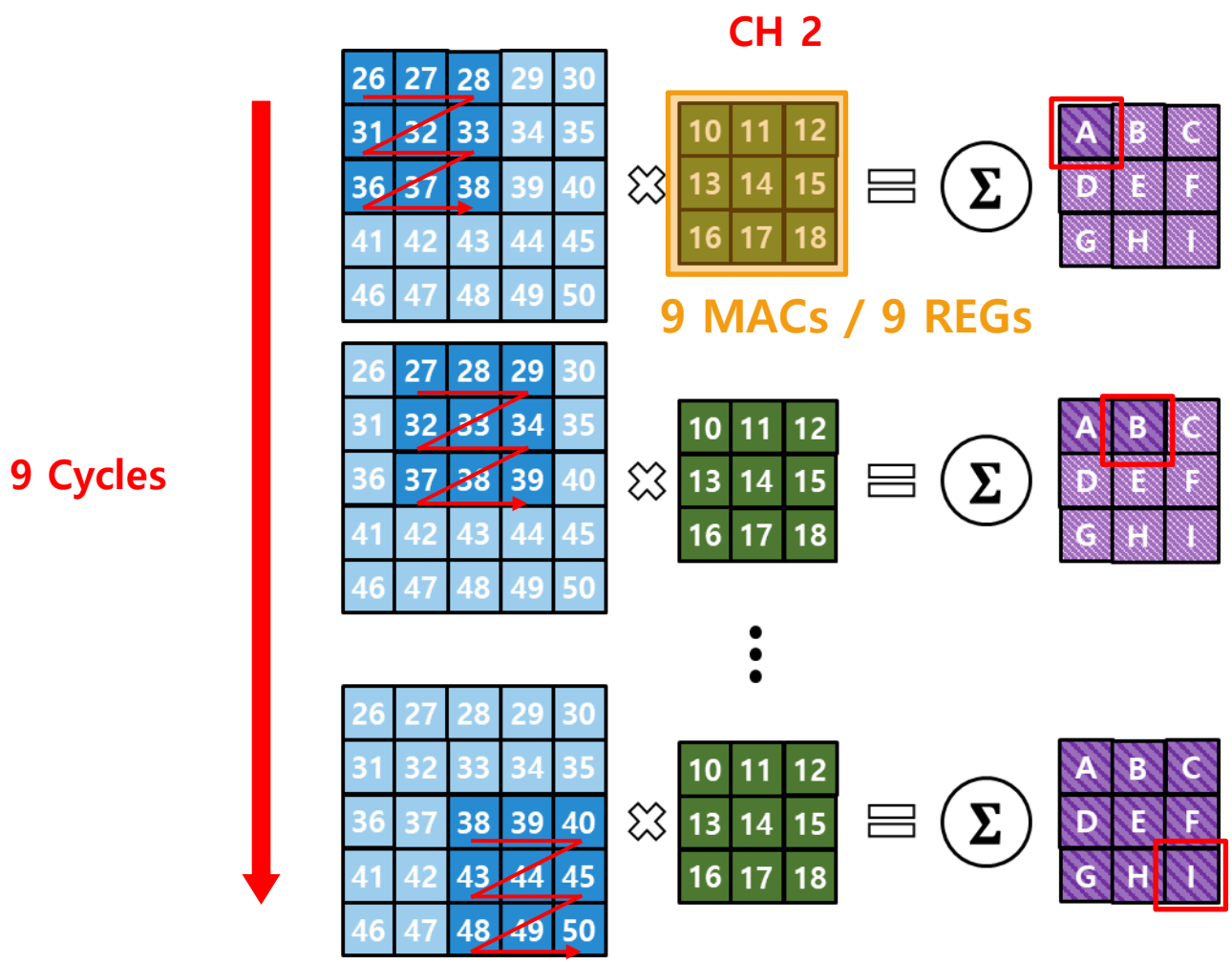


<Conv2d Naive Output Stationary>

Output Stationary (3)

◈ Naive Output Stationary(9MACs/9REGs)

A : Not Started
A : Partial Sum CH1
A : Partial Sum CH2
A : Partial Sum CH3



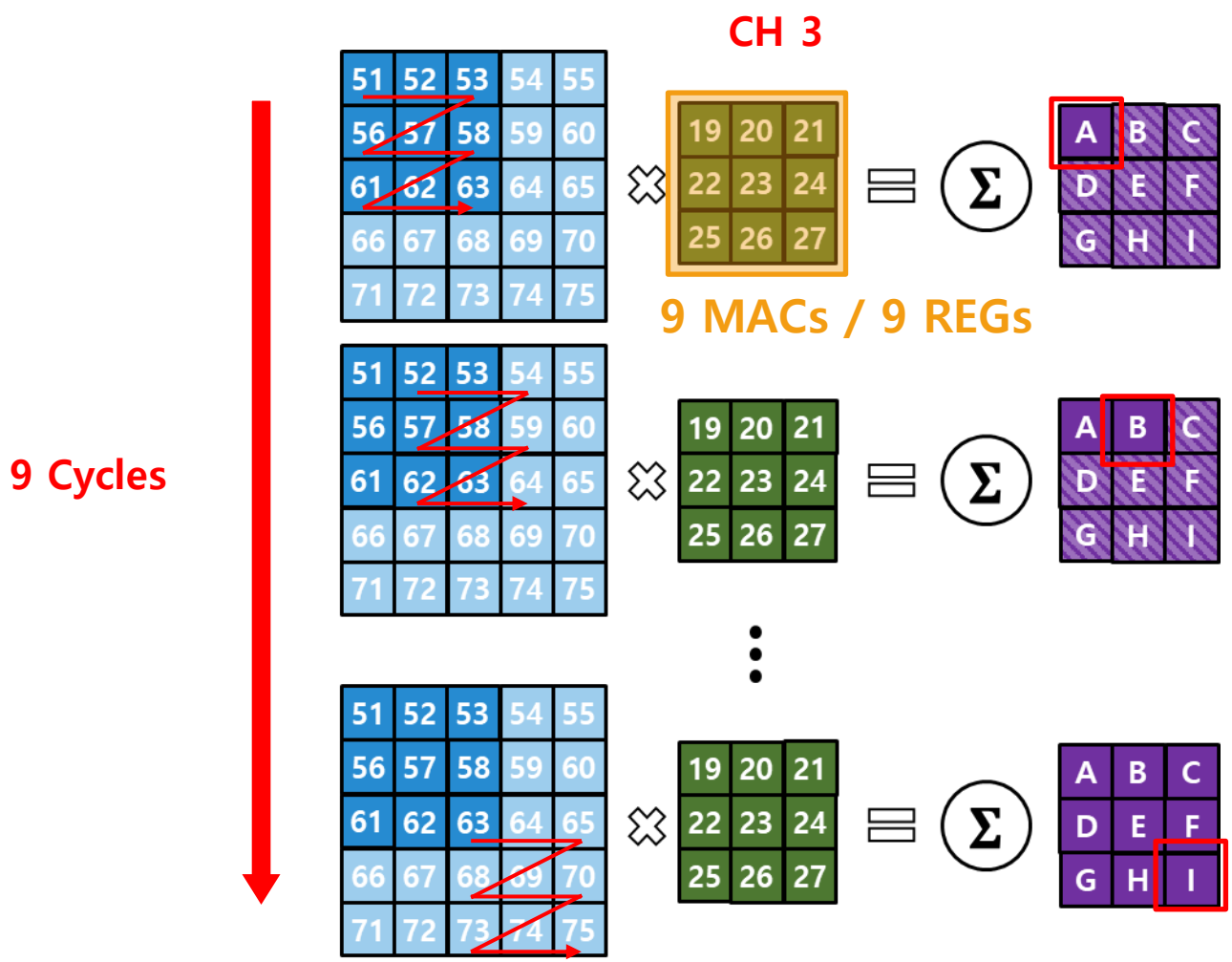
34+34 Clocks

<Conv2d Naive Output Stationary>

Output Stationary (4)

◈ Naive Output Stationary(9MACs/9REGs)

A : Not Started
A : Partial Sum CH1
A : Partial Sum CH2
A : Partial Sum CH3



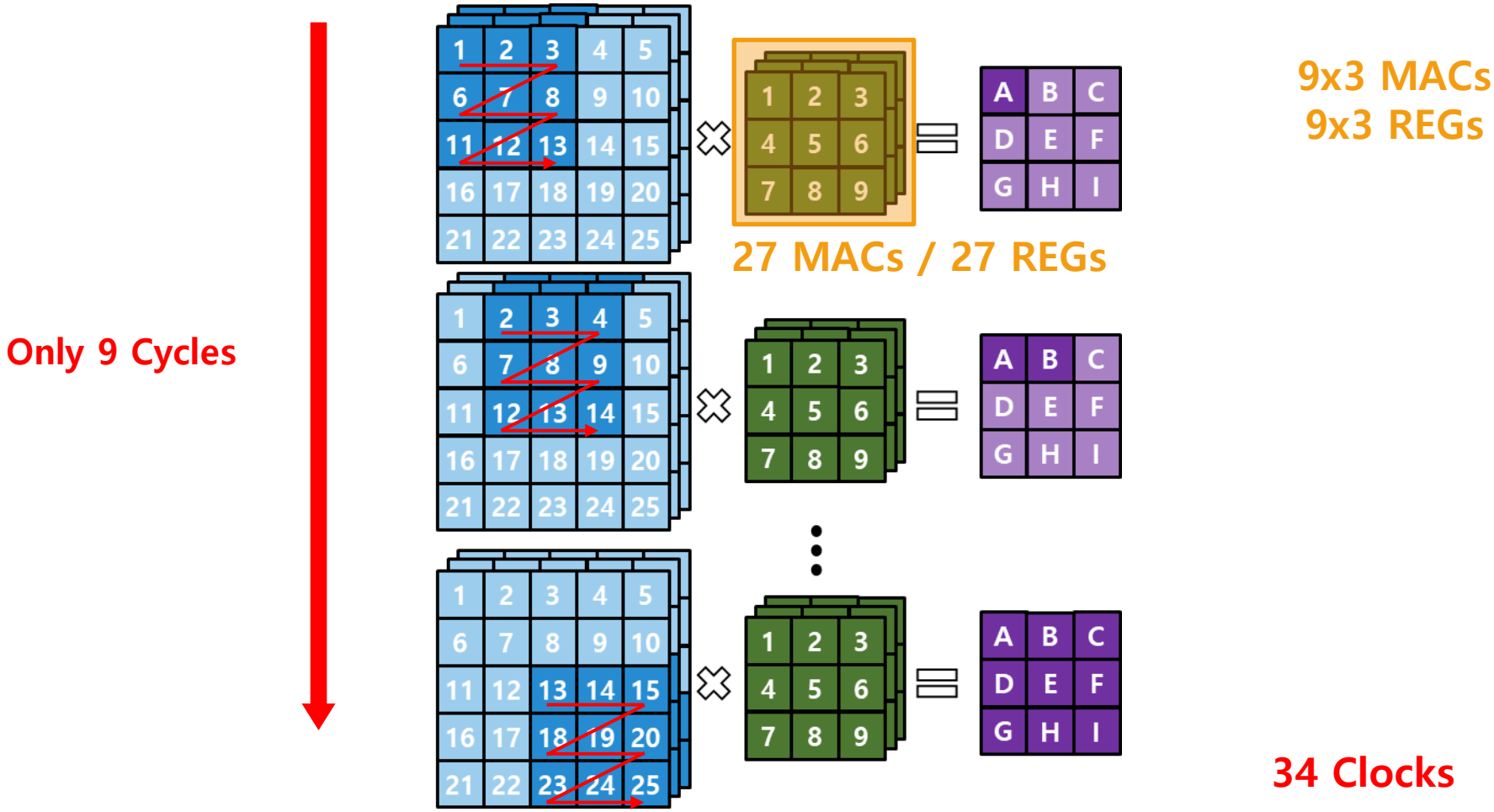
34+34+34 Clocks
=102 Clocks

<Conv2d Naive Output Stationary>

Output Stationary (5)

Channel-wise Parallelism(27MACs/27REGs)

- **PE array is expanded to match the channel size for parallel processing**.
- Total latency is reduced through parallel processing.



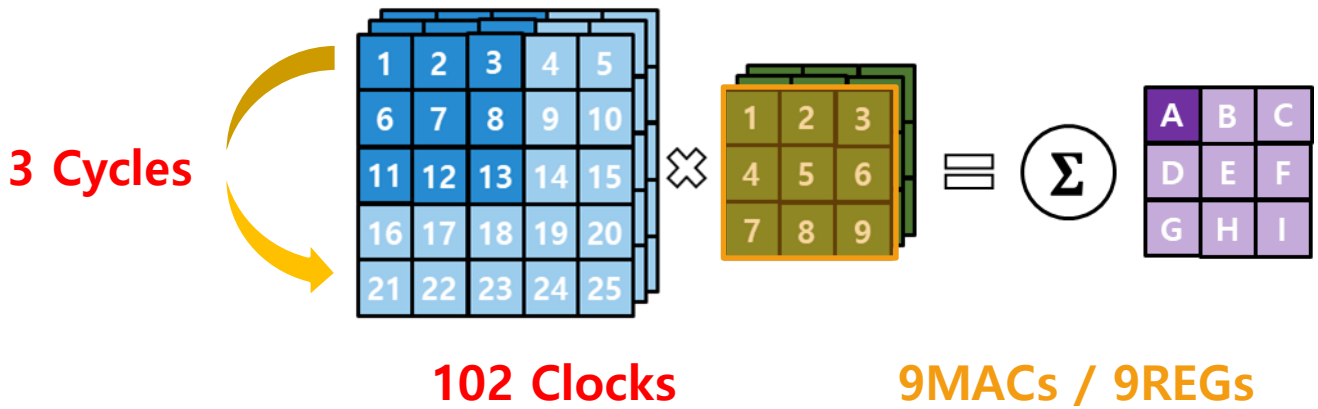
<Conv2d Channel-wise Parallelism>

Output Stationary (6)

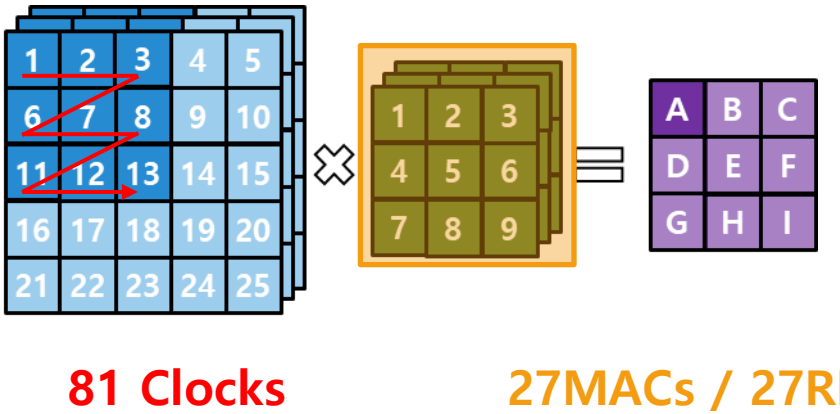
Naive Output Stationary vs Channel-wise Parallelism

: dataflow

: PE



<Conv2d Naive Output Stationary>



<Conv2d Channel-wise Parallelism>

Thank You!